WP module
Learning Optimization
Week 6
Dynamic Programming 1

2024-11-12
Dr. Alwin Haensel

BHT
Berliner Hochschule für Technik

We have an initial demand model.

In week 4, we fitted linear regressions to specific sections in the selling_season, e.g. selling_periods 1..20. We then obtain a linear function which can be used to estimate the demand for given own price and competitor price.

```
def est_demand(price,
               comp_price,
               para = [-0.0564, 0.0133, 3.676]):
    """
    parameter from OLS (last week):
    price                -0.056444
    price_competitor      0.013317
    intercept             3.675965
    """
    demand = para[0]*price + para[1]*comp_price + para[2]
    demand = max(0,demand)
    return round(demand,1)
```

$$demand = \beta_{price} * price_{own} + \beta_{comp\_price} * price_{competitor} + \beta_{intercept}$$

What is my optimal price?

$$demand = \beta_{price} * price_{own} + \beta_{comp\_price} * price_{competitor} + \beta_{intercept}$$

$$revenue = demand * price_{own}$$

=> revenue is a nice quadratic function in the price

What is my revenue maximizing price?

What is my optimal price?

$$demand = \beta_{price} * price_{own} + \beta_{comp\_price} * price_{competitor} + \beta_{intercept}$$
$$revenue = demand * price_{own}$$
$$revenue(price_{own}) = \beta_{price} * {price_{own}}^2 + \beta_{comp\_price} * price_{competitor} * price_{own} + \beta_{intercept} * price_{own}$$

$$\frac{\nabla\, revenue}{\nabla price_{own}}(price_{own}) = 2 * \beta_{price} * price_{own} + \beta_{comp\_price} * price_{competitor} + \beta_{intercept}$$

$$\frac{\nabla\, revenue}{\nabla price_{own}}(price_{own}) = 0$$

$$\boldsymbol{{price_{own}}^{opt}} = \frac{-\beta_{comp\_price} * price_{competitor} - \beta_{intercept}}{2 * \beta_{price}}$$

$$\frac{\nabla\, revenue}{\nabla^2 price_{own}}(price_{own}) = 2 * \beta_{price} < 0$$

Let‘s try to use that optimal price in every time period …

```
t=1, price=38.5, demand=2.2, revenue=83.55, remaining_capacity=17.8
t=2, price=38.5, demand=2.2, revenue=167.09, remaining_capacity=15.7
t=3, price=38.5, demand=2.2, revenue=250.63, remaining_capacity=13.5
t=4, price=38.5, demand=2.2, revenue=334.18, remaining_capacity=11.3
t=5, price=38.5, demand=2.2, revenue=417.73, remaining_capacity=9.1
t=6, price=38.5, demand=2.2, revenue=501.27, remaining_capacity=7.0
t=7, price=38.5, demand=2.2, revenue=584.82, remaining_capacity=4.8
t=8, price=38.5, demand=2.2, revenue=668.36, remaining_capacity=2.6
t=9, price=38.5, demand=2.2, revenue=751.90, remaining_capacity=0.5
t=10, price=38.5, demand=0.5, revenue=770.00, remaining_capacity=0.0
```

- capacity = 20
- consider 10 time periods for selling
- OLS parameter for price, competitor_price and intercept are [-0.0564, 0.0133, 3.676]
- assume constant competitor price of 50

Optimal price in every time period = 38.5 **=>** **revenue = 770**

… Now use price 42 instead of the „myopic“ price 38.5

- capacity = 20
- consider 10 time periods for selling
- OLS parameter for price, competitor_price and intercept are [-0.0564, 0.0133, 3.676]
- assume constant competitor price of 50

```
t=1, price=42.0, demand=2.0, revenue=82.82, remaining_capacity=18.0
t=2, price=42.0, demand=2.0, revenue=165.65, remaining_capacity=16.1
t=3, price=42.0, demand=2.0, revenue=248.47, remaining_capacity=14.1
t=4, price=42.0, demand=2.0, revenue=331.30, remaining_capacity=12.1
t=5, price=42.0, demand=2.0, revenue=414.12, remaining_capacity=10.1
t=6, price=42.0, demand=2.0, revenue=496.94, remaining_capacity=8.2
t=7, price=42.0, demand=2.0, revenue=579.77, remaining_capacity=6.2
t=8, price=42.0, demand=2.0, revenue=662.59, remaining_capacity=4.2
t=9, price=42.0, demand=2.0, revenue=745.42, remaining_capacity=2.3
t=10, price=42.0, demand=2.0, revenue=828.24, remaining_capacity=0.3
```

Revenue = 828 => We need to consider future potential revenue when making our pricing decision.

Dynamic Programming – in short

In Dynamic Programming we often speak of a value function at time t and system state x and denote it with $V_t(x)$.

$$\boldsymbol{V_t(x) = Max_{price\ p}\ E[\ V_{t-1}(x - demand(p)) + demand(p) * p\ ]}$$

In our case think about it like this:

- x = remaining capacity
- t = time period for selling, but we are counting down now
    - t=10 -> we have still 10 time period to sell, now plus 9 future period
    - t=1 -> we are in the last period
- $V_t(x)$ is the revenue value of having x remaining capacity free at time periods t
- $V_0(x) = 0$ for all capacity levels x

Dynamic Programming – even simpler ;-)

Now let us simplify the value function by extending it with the parameter d for demand -> $V_t(x, d)$.

$$\boldsymbol{V_t(x, d) = V_{t-1}(x-d) + d * price(d)}$$

Assume price(d) is the reverse function of the demand-price relation ship, meaning for a given target demand, we can derive the "best price" to generate this demand.

Notebook task: Define the function **get_best_price_target_demand** (=price(d)), which generated the best price for a target demand.

```
get_best_price_target_demand(target_demand=3)
```

```
(24.0, 3)
```

Dynamic Programming – even simpler ;-)

We have $\boldsymbol{V_t(x,d) = V_{t-1}(x-d) + d * price(d)}$

Note that $\boldsymbol{V_{t-1}(x-d)}$ *is referring to the value of the best action at time t-1 with remaining capacity x-d.*

$$\boldsymbol{V_{t-1}(x-d) = max\ \ V_{t-1}(x,d)}$$

We can compute all possible actions at $\boldsymbol{V_{t-1}(x,d)}$ *and hence know (just look up) the best.*

Full Value Function Lookup Table

We can actually compute all possible values of $V_t(x, d)$ recursively starting at

- $V_0(x, d)$ **for all x, d**
- $V_1(x, d)$ **for all x, d**
- **…**

Given our small problem dimension: (t = 100..1) x (x=80..0) x (d=0..**5**) $\approx$ 40k

```
df_V
```

```
100%|██████████████████████████████████████████
```

	t	free_cap	V_t	price_t	demand_t
0	0.0	0.0	0.0	0.0	0.0
1	0.0	1.0	0.0	0.0	0.0
2	0.0	2.0	0.0	0.0	0.0
3	0.0	3.0	0.0	0.0	0.0
4	0.0	4.0	0.0	0.0	0.0
...	...	...	...	...	...
39076	100.0	80.0	4661.0	77.0	0.0
39077	100.0	80.0	4661.0	59.0	1.0
39078	100.0	80.0	4626.0	41.5	2.0
39079	100.0	80.0	4556.0	24.0	3.0
39080	100.0	80.0	4449.0	6.0	4.0

39081 rows × 5 columns